

Schéma relationnel et normalisation

Guillaume Guex guillaume.guex@unil.ch

Schéma relationnel

Le schéma relationnel

La dernière étape de la modélisation logique d'une base de donnée est de construire son **schéma relationnel**. Celui-ci se dérive à partir du diagramme ER.

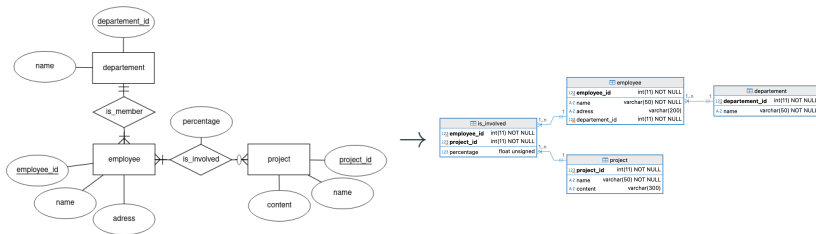


Schéma relationnel : étapes de création

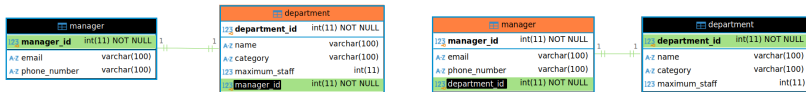
Les différentes étapes pour la création du schéma relationnel à partir du diagramme ER sont les suivantes :

1. Créer des **tables** pour toutes les entités.
2. Créer les **attributs** de ces tables, leur assigner un type et d'éventuelles contraintes.
3. Assigner les **clés primaires**.
4. Résoudre les **associations** et créant des clés étrangères, des tables d'association et éventuellement en créant de nouveaux attributs.

Les trois premières étapes sont assez directes (en démo). Nous allons nous concentrer ici sur la résolution des associations.

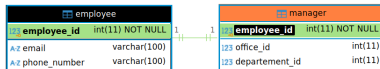
Associations un-à-un

Lorsqu'une **association un-à-un** existe, on peut la modéliser avec une clé étrangère dans l'une des deux tables :



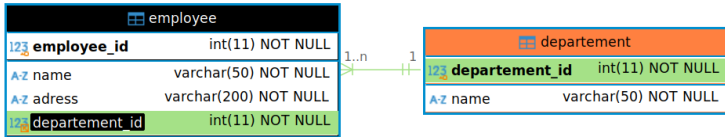
La valeur de cette clé doit être déclarée **UNIQUE** afin de représenter l'unicité du lien. Les associations obligatoires sont désignées d'un côté avec **NOT NULL** (elles sont par défaut facultatives).

Si une entité est un **sous-ensemble** d'une autre, on peut aussi utiliser **sa clé primaire comme clé étrangère** :



Associations un-à-plusieurs

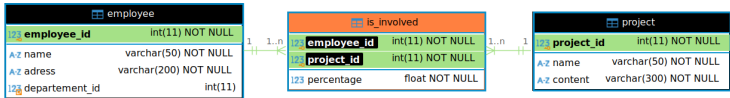
Dans une **association un-à-plusieurs**, c'est l'entité qui a une référence unique qui doit posséder la clé étrangère :



L'association unique peut être obligatoire en la déclarant **NOT NULL** (la relation multiple est forcément facultative).

Associations plusieurs-à-plusieurs

Les **associations plusieurs-à-plusieurs** doivent être modélisée avec une **tables d'association** :



La clé primaire de ce tableau est souvent constitué des deux clés étrangères formant l'association.

La normalisation

La **normalisation** est un processus qui consiste en la restructuration d'une base de données afin que cette dernière respecte une certaine **forme normale**.

Ce processus permet d'assurer une certaine forme d'**intégrité** aux données et d'**éviter des redondances**.

La normalisation : pourquoi ?

Prenons un exemple de table non-normalisée :

E#	Name	Street	City	D#	DepartmentName
E19	Stewart	E Main Street	Stow	D6	Accounting
E1	Murphy	Morris Road	Kent	D3	IT
E7	Howard	Lorain Avenue	Cleveland	D5	HR
E4	Bell	S Water Street	Kent	D6	Accounting

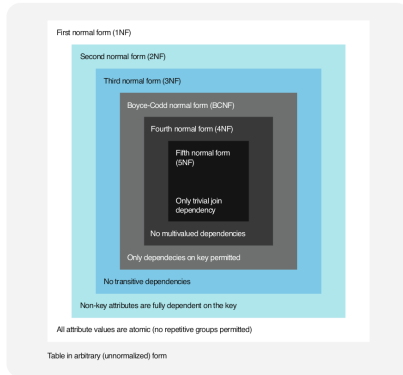
Source : Meier, A., & Kaufmann, M. (2019). *SQL & NoSQL databases*. Wiesbaden : Springer.

Des données sont redondantes, mais cela peut aussi créer des **anomalies** :

- Anomalies d'**insertion** (p.ex. création d'un département vide).
- Anomalies de **suppression** (p.ex. suppression de tous les employés d'un département).
- Anomalies de **mise à jour** (p.ex. changement de nom d'un département).

Les formes normales

Il existe de nombreuses **formes normales** (jusqu'à neuf) qui sont de plus en plus contraignantes :



Source : Meier, A., & Kaufmann, M. (2019). *SQL & NoSQL databases*. Wiesbaden : Springer.

Dans la pratique, une BD qui respecte la forme 3NF est dite normalisée.

Première forme normale : 1NF

Une table respecte la **première forme normale** si on peut **construire une clé primaire** et que tous ses attributs possèdent une **valeur atomique**.

E#	Name	City	P#
E7	Howard	Cleveland	[P1, P9]
E1	Murphy	Kent	[P7, P11, P9]

E#	P#	Name	City
E7	P1	Howard	Cleveland
E7	P9	Howard	Cleveland
E1	P7	Murphy	Kent
E1	P11	Murphy	Kent
E1	P9	Murphy	Kent

Source : Meier, A., & Kaufmann, M. (2019). *SQL & NoSQL databases*. Wiesbaden : Springer.

Passer à la première forme normale génère généralement des redondances.

Pour comprendre les deuxième et troisième formes normales, il faut comprendre le principe de **dépendance fonctionnelle**.

On dit qu'un attribut B **dépend fonctionnellement** de l'attribut A si, pour une valeur donnée de A , il n'existe qu'une seule valeur possible de B . On le note $A \rightarrow B$.

Exemples :

- `employee_id` $\rightarrow$ `employee_name`.
- `employee_name` $\rightarrow$ `city`.
- `departement` $\rightarrow$ `manager`.

Deuxième forme normale : 2NF

Une table respecte la **deuxième forme normale** si elle est 1NF et :

- Il n'y a qu'un seul attribut comme clé primaire OU
- Les attributs non-clés dépendent fonctionnellement de toute la clé, mais d'aucune de ses parties.

PROJECT EMPLOYEE (first normal form)

E#	P#	Name	City
E7	P1	Howard	Cleveland
E7	P9	Howard	Cleveland
E1	P7	Murphy	Kent
E1	P11	Murphy	Kent
E1	P9	Murphy	Kent

EMPLOYEE (2NF)

E#	Name	City
E7	Howard	Cleveland
E1	Murphy	Kent

INVOLVED (2NF)

E#	P#
E7	P1
E7	P9
E1	P7
E1	P11
E1	P9

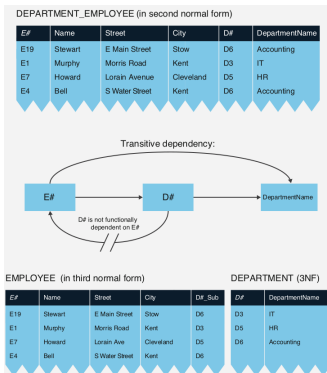
Source : Meier, A., & Kaufmann, M. (2019). *SQL & NoSQL databases*. Wiesbaden : Springer.

La redondance est ainsi fortement réduite.

Troisième forme normale : 3NF

Une table respecte la **troisième forme normale** si elle est 2NF et qu'il n'existe pas de **dépendance transitive** entre un attribut non-clé et une clé.

Il y a dépendance transitive entre une clé et un attribut B si :
clé $\rightarrow A$, $A \rightarrow B$, mais pas $A \rightarrow$ clé.



Cette forme permet de réduire le risque d'avoir des anomalies.

Démo : création d'un schéma relationnel